CSCI2100B/ESTR2102B Data Structures (Fall 2024)

Lab Assignment #1
Schedule: Week 2
1. To familiarize with the implementation of linked lists and arraylists 2. To familiarize the list abstract data type

Submission guideline (Online judge)

1. Name your program(s) according to the question (e.g. `lab1-q1.c`)
2. Write your programs under "`Desktop/mywork/lab1`"
3. Duplicate your programs before submission
4. Submit your programs under "`Desktop/submission/lab1`"
5. Check the submission folder for your grading report (Be patience, auto grading need some time)
6. Resubmit your work if you cannot receive full score
7. Visit "`Other Locations/Computer/2100share/lab1`" in the VM to download the starting programs
8. Version of the gcc compiler in the online judge: C99
9. Deadline: 2 October, 2024 (Wednesday)
10. Late penalty is 30%, and the judge for lab 1 is closed on 9 October, 2024 (Wednesday).

Grading scheme (CSCI2100B)

1. Total score is 100%
2. The weighting of each question is marked on the question
3. Bonus score is at most 13%

Requirements for ESTR2102B: Non-zero scores for the extra question

Useful commands

1. Compile your program

```
gcc -o prog labX-q1.c labX-q1-tc1.c
gcc -o prog labX-q3.c
```

2. Run the program with arguments

```
./prog inputfile.txt outputfile.txt
```

Question 1: Linked list ADT (40%)

You are going to implement the linked list abstract data type for floating point number. The header file “lab1-q1.h” and the description of the functions are as follows.

Your program should not contain main(). Name your program as “lab1-q1.c”.

```
// lab1-q1.h

// Initialize the linked list as an empty list #1
// An empty list is defined as a null pointer
void list_init_empty(Node **list);

// Initialize the linked list with an array of n elements #2
// The last element of the arr is the first element in the linked list
void list_init(Node **list, double *arr, int n);

// Insert value to the front of the linked list #3 #4
// Return the head of the linked list
Node* list_insert(Node *list, double value);

// Delete value from the linked list #5 #6 #7
// If value is in the list, then delete it
// Note: there can be multiple value, you need to delete all of them
// If value is not in the list, then do nothing
// Return the head of the linked list
Node* list_delete(Node *list, double value);

// Check if value exists in the list or not #8
// If value is in the list, return 1
// If value is not in the list, return 0
int list_contain(Node *list, double value);

// Free the list, if list is not NULL #9
// Assign NULL to the *list
void list_free(Node **list);

// The print function print the list in an output string #10
// Example: "14.00 15.00 17.50 "
// You can assume the number of nodes is not more than 100
// You can assume the largest value is capped by 100
char* list_print(Node *list);
```

Sample Test Case (lab1-q1-tc1.c):

```
// This file is named as lab1-q1-tc1.c
#include"lab1-q1.h"
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<limits.h>
```

```
int main(int argc, char *argv[]){

	FILE *fout;
	Node *list;
	int isCorrect, i;
	fout = fopen(argv[1], "w");

	// default, your program is correct
	isCorrect = 1;

	// check the function, list_init_empty()
	list_init_empty(&list);

	// if list_init_empty() is not implemented appropriately
	if (list != NULL) {
		isCorrect = 0;
	}

	// output "Correct" if test case is passed
	if (isCorrect == 1) {
		fprintf(fout, "List: Correct\n");
	} else {
		fprintf(fout, "List: Wrong Answer\n");
	}
	fclose(fout);

	return 0;
}
```

Question 2: Arraylist ADT (30%)

You are going to implement the arraylist abstract data type for words using a 2-dimensional array. The header file “lab1-q2.h” and the description of the functions are as follows. **(Hint: This is a circular list!)**
You can assume that each word (alphabet only) is no longer than 50 characters. The maximum size of the list is 100. Your program should not contain `main()`. Name your program as “lab1-q2.c”.

```
// lab1-q2.h
// Cannot modify this file
typedef struct _list {
	char word[100][51];	/* The 2D array for words */
	int first, last;  /* Index for the first and last position */
} ArrayList;

/* Initialize the arraylist as an empty list #1 */
/* first and last are set to 0 */
void arraylist_init_empty(ArrayList **list);
```

```
/* Initialize the arraylist with an array of n words #2 */
/* The last element of the arr is the last element in the list */
/* Suppose n < 100 */
void arraylist_init(ArrayList **list, char arr[100][51], int n);

/* Insert value to the end of the list #3 #4 */
/* If list is full, then do nothing */
void arraylist_insert(ArrayList *list, char *word);

/* Delete the first word from the list #5 #6 */
/* If list is empty, then do nothing and return null pointer */
/* Return the deleted word for non-empty list */
char* arraylist_deletefirst(ArrayList *list);

/* Delete the last word from the list #7 #8 */
/* If list is empty, then do nothing and return null pointer */
/* Return the deleted word for non-empty list */
char* arraylist_deletelast(ArrayList *list);

/* Return the size of the list #9 */
/* Note: The return value ranges from 0 and 99 inclusively */
int arraylist_size(ArrayList *list);

/* Free the list, if list is not NULL #10 */
/* Assign NULL to the *list */
void arraylist_free(ArrayList **list);

/* The print function print the list in an output string #11 */
/* Example: "haha Hello Yes " */
/* You can assume the number of nodes is not more than 99 */
char* arraylist_print(ArrayList *list);
```

Sample Test Case (lab1-q2-tc1.c):

```
// This file is named as lab1-q2-tc1.c
#include"lab1-q2.h"
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<limits.h>

int main(int argc, char *argv[]){

      FILE *fout;
      ArrayList *list;
      int isCorrect, i;
      fout = fopen(argv[1], "w");

      // default, your program is correct
      isCorrect = 1;
```

```
	// check the function, arraylist_init_empty()
	arraylist_init_empty(&list);

	// if arraylist_init_empty() isn't implemented appropriately
	if (list == NULL) {
		isCorrect = 0;

	} else if (list->size != 0) {
		isCorrect = 0;
	} else if (list->word[0][0] != 0 || list->word[99][50] != 0) {
		isCorrect = 0;
	}

	// output "Correct" if test case is passed
	if (isCorrect == 1) {
		fprintf(fout, "List: Correct\n");
	} else {
		fprintf(fout, "List: Wrong Answer\n");
	}
	fclose(fout);

	return 0;
}
```

Question 3: Josephus problem (30%)

People are standing in a circle waiting to be executed. Counting begins at a specified point in the circle and proceeds around the circle in a specified direction. Write a program to simulate the Josephus problem.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as “lab1-q3.c”.

Input file: Two positive integers, “n k”. n is the number of people standing in the circle, while k is the count for each step (skipping k-1 people and the k-th is executed).

Output file: A line of n integers (1-based) separated by space. The sequence of integers indicates the order of execution.

Sample Input:

```
5 3
```

Sample Output:

```
3 1 5 2 4
```

Question 4: Music player (Bonus 13%)

You are asked to simulate a music player. The music player is initialized with time = 0 min and one default song = “Greensleeves” with duration 5 min. This song can never be removed from the music player. The player will play all songs in the list order and repeat again and again. There are 5 operations available for the music player.

Operations	Description
`Add <string> <int> <int>` `E.g. Add BadRomance 4 5`	Add the song BadRomance with duration 4 min at time = 5 min to the end of playlist
`Insert <string> <string> <int> <int>` `E.g. Insert Greensleeves HA 1 20`	Insert the song HA with duration 1 min after Greensleeves at time = 20 min. If HA already exists in the playlist, then HA can appear more than once in the playlist. If Greensleeves does not exist in the playlist, then HA will be inserted at the end of the playlist. If there are multiple Greensleeves in the playlist, then HA is inserted after the first Greensleeves.
`Delete <string> <int>` `E.g. Delete BadRomance 40`	Delete the song BadRomance at time = 40 min If there are more than 1 copy of BadRomance, all BadRomance will be removed. The first Greensleeves can never be deleted. If a song is deleted while playing, the player will delete the song after playing this.
`Query <int>` `E.g. Query 5`	Query what the song is playing at time = 5 min
`Print <int>` `E.g. Print 21`	Print all the songs in the list at time = 21 min

In this problem, you can also assume the operation is sorted by the time.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as “lab1-q4.c”.

Input file: Each line contains one operation in the table above. You can assume that the name of the song contains capital or small letters only. The length of a song will not be longer than 100.

Output file: Output the results based on the `query` and `print` operations. You can assume the maximum number of songs in the playlist does not exceed 100.

Sample Input:

```
Add BadRomance 4 5
Query 5
Insert Greensleeves HA 1 20
Query 20
Print 21
Delete BadRomance 40
Query 42
Print 42
Query 43
Print 43
```

Sample Output:

```
Greensleeves
BadRomance
Greensleeves HA BadRomance
BadRomance
Greensleeves HA BadRomance
Greensleeves
Greensleeves HA
```

Note: If you have a trailing space at the end of line, you will receive “wrong answer”. Take this sample output as an example, if it is written in a string, it will be,

```
"Greensleeves\nBadRomance\nGreensleeves HA
BadRomance\nBadRomance\nGreensleeves HA
BadRomance\nGreensleeves\nGreensleeves HA\n"
```